LAB 06 / LOCAL AI

Local AI with Ollama and Open WebUI

Windows setup, container deployment, and browser access

Cristian K. Leung · Academic Lab Portfolio

Executive Summary

Explored a local language-model workflow using Ollama and a browser interface. Installation screenshots, container views, and troubleshooting notes document the Windows environment and the steps taken to make the model accessible through a GUI.

> **SKILLS DEMONSTRATED**
>
> Ollama · Docker Desktop · PowerShell · WSL · Open WebUI

- Installed Ollama and documented local model interaction.
- Worked through Docker Desktop and WSL setup on Windows.
- Explored a browser UI and optional custom-interface files.

Evidence Included

Installer and model screenshots, PowerShell output, Docker container views, browser UI, and custom-interface examples.

A complete CPU-only reference deployment now uses two containers on one Docker network, persistent data volumes, and an explicit Ollama backend URL. Original installation screenshots remain evidence of the earlier Windows exercise.

Objective

The objective was to run a language model locally and explore a browser-based interface that makes the model easier to access.

Technical Background

Large language models generate text from patterns learned during training. Running a model locally changes where inference takes place and allows direct control of model files and the serving environment.

Local inference can reduce reliance on an external service for model requests. Privacy still depends on the model, application settings, network connections, and any external features enabled in the interface.

The lab combines a local model runtime with a browser interface. It focuses on installation, container setup, and troubleshooting rather than model training.

Ollama simplifies downloading and running supported language models. It provides a local interface and service that other applications can use.

Docker packages applications and their dependencies in containers. Docker Desktop provides the container environment used in this Windows exercise.

PowerShell provides the command-line interface used for setup. Windows Subsystem for Linux (WSL) supports the Docker Desktop configuration shown in the lab.

Open WebUI provides a browser interface for interacting with model services. The exercise also explores optional custom interface files.

Procedure and Evidence

Download Ollama from https://ollama.com/.

Select the installer for the operating system.

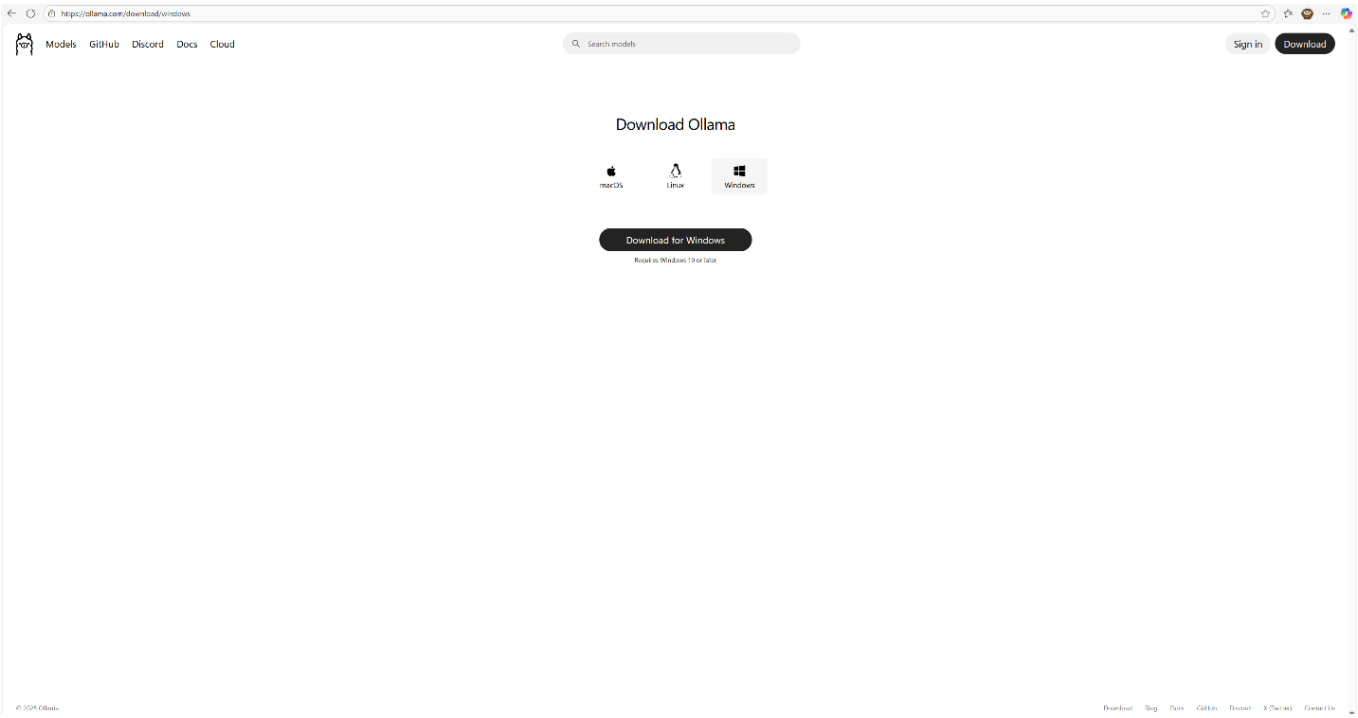


Figure 01. Select the installer for the operating system.

Open the downloaded installer.

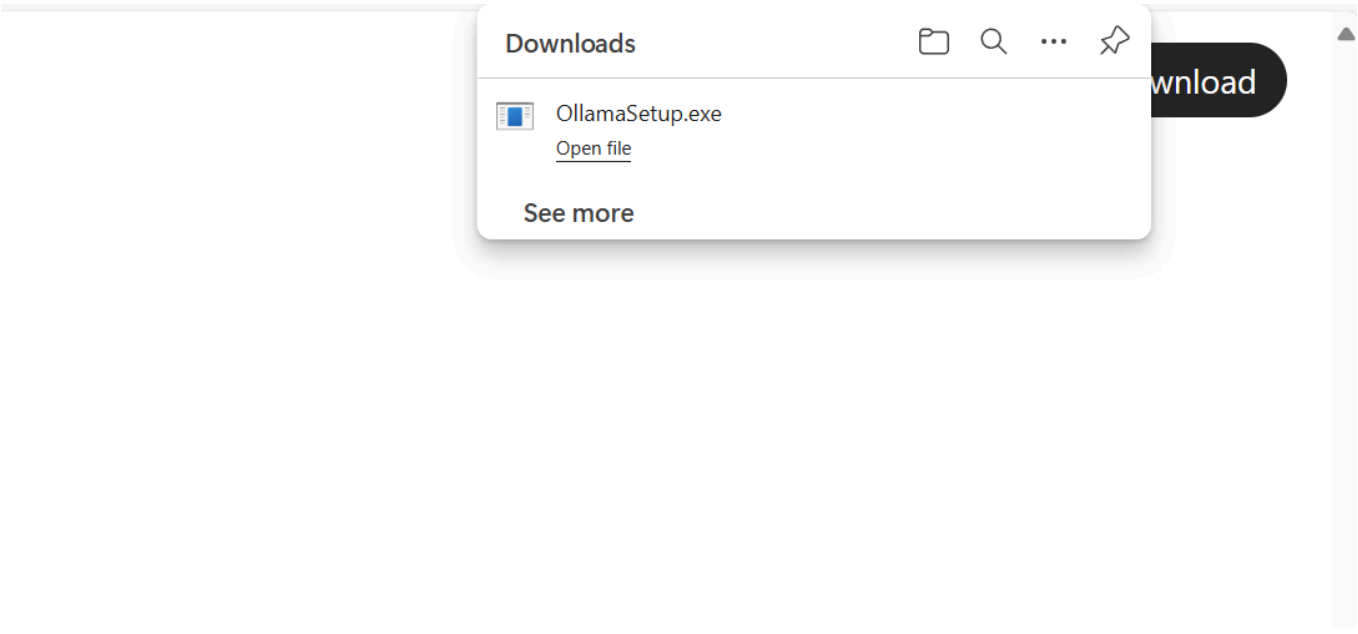


Figure 02. Open the downloaded installer.

Run the installer and complete the setup.

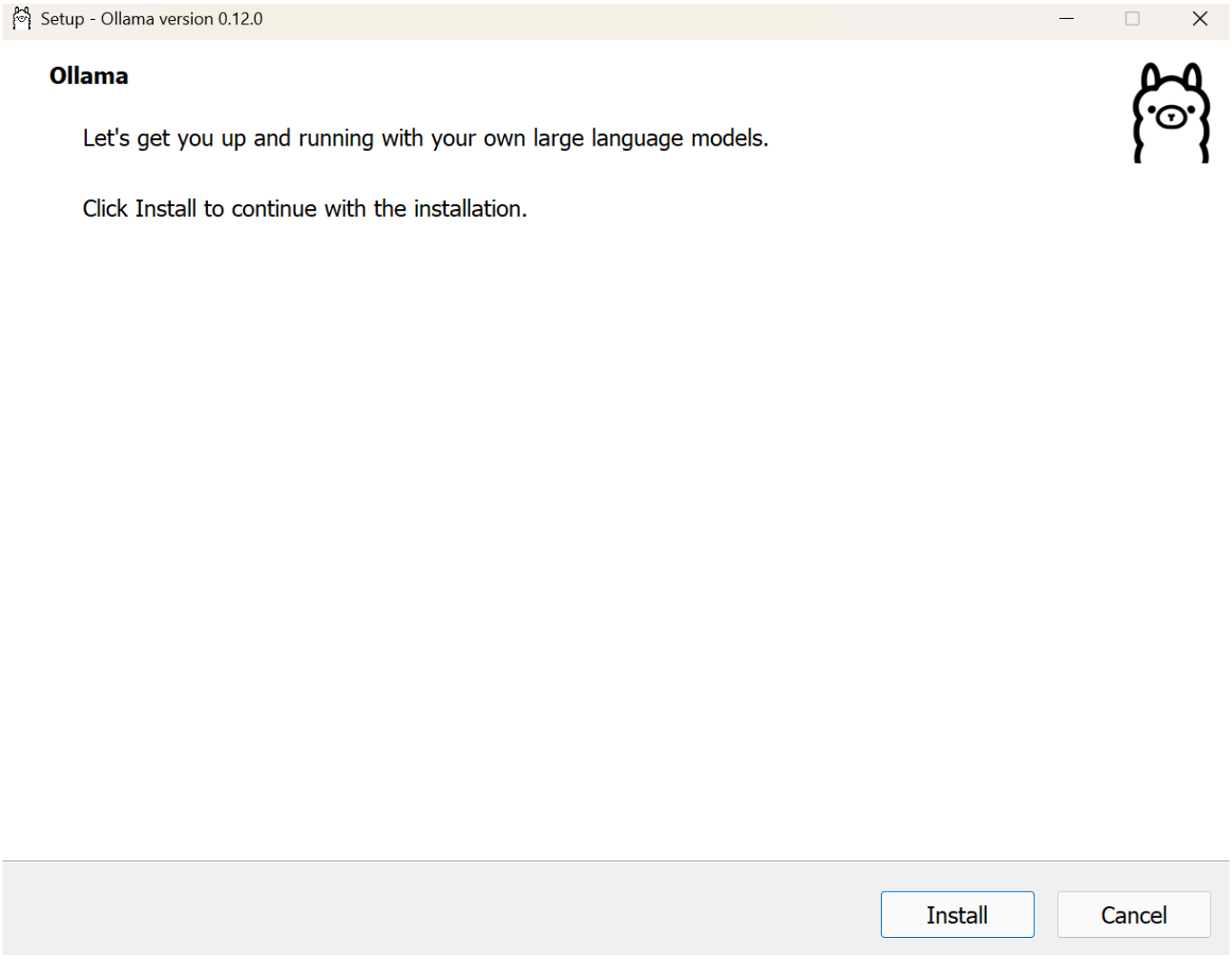


Figure 03. Run the installer and complete the setup.

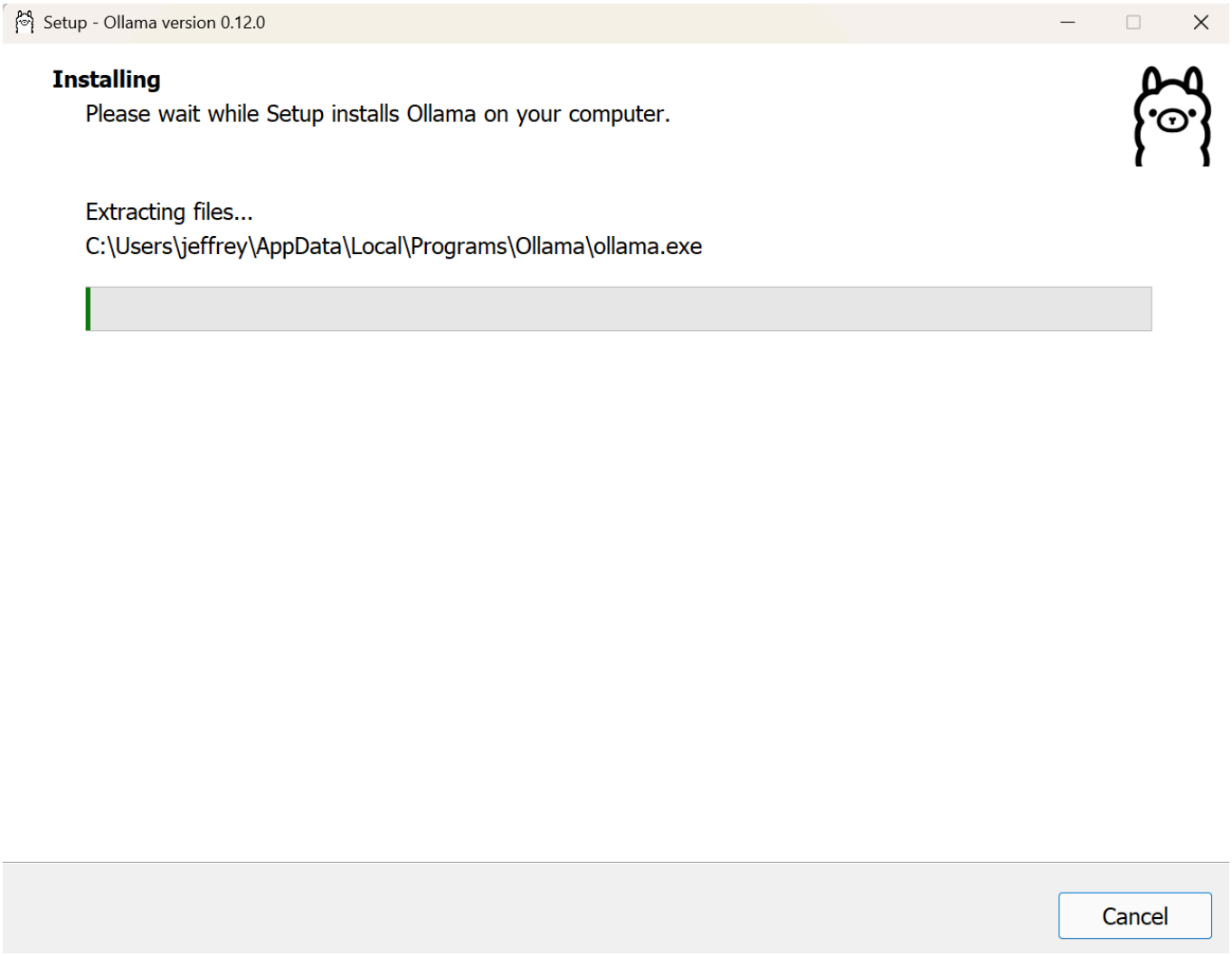


Figure 04. Run the installer and complete the setup.

Launch Ollama and select a model. The screenshots document the initial application and model interaction.



Figure 05. Launch Ollama and select a model. The screenshots document the initial application and model interaction.

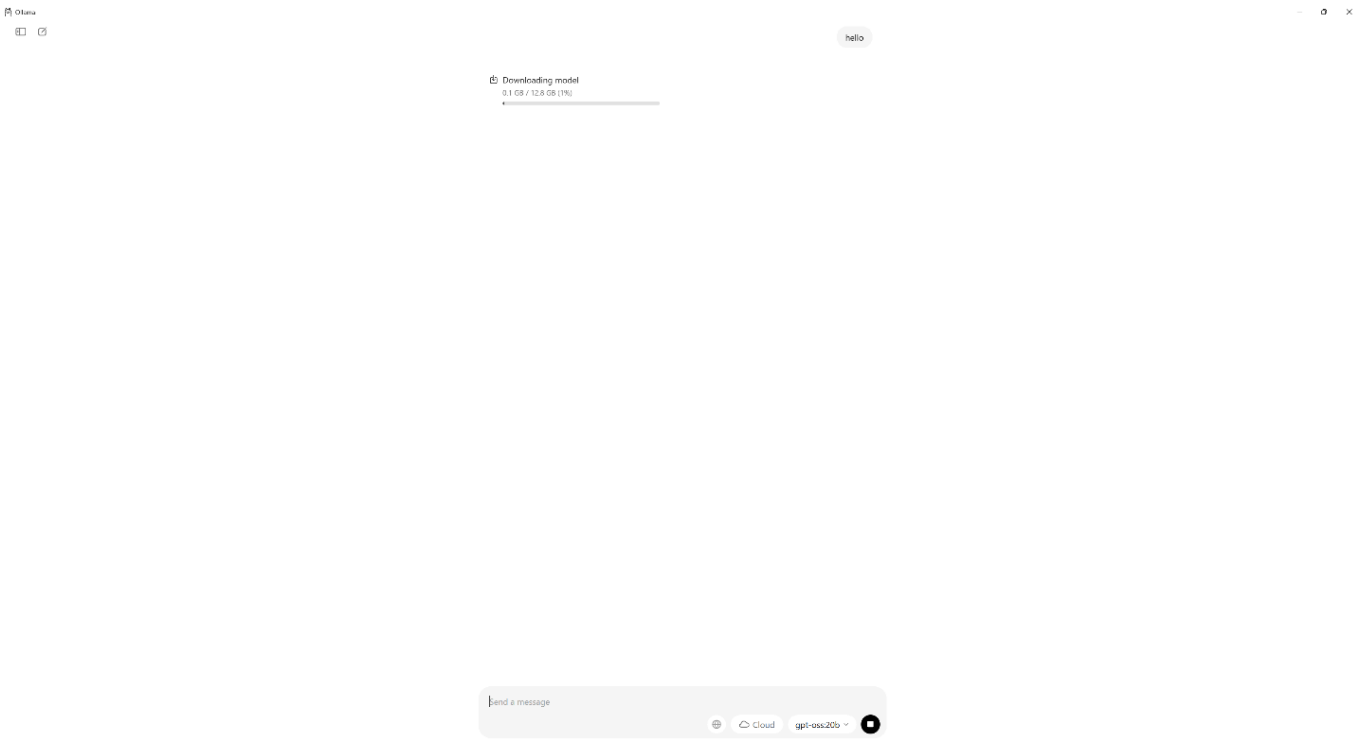


Figure 06. Launch Ollama and select a model. The screenshots document the initial application and model interaction.

Download Docker Desktop from https://www.docker.com/ while the model setup completes.

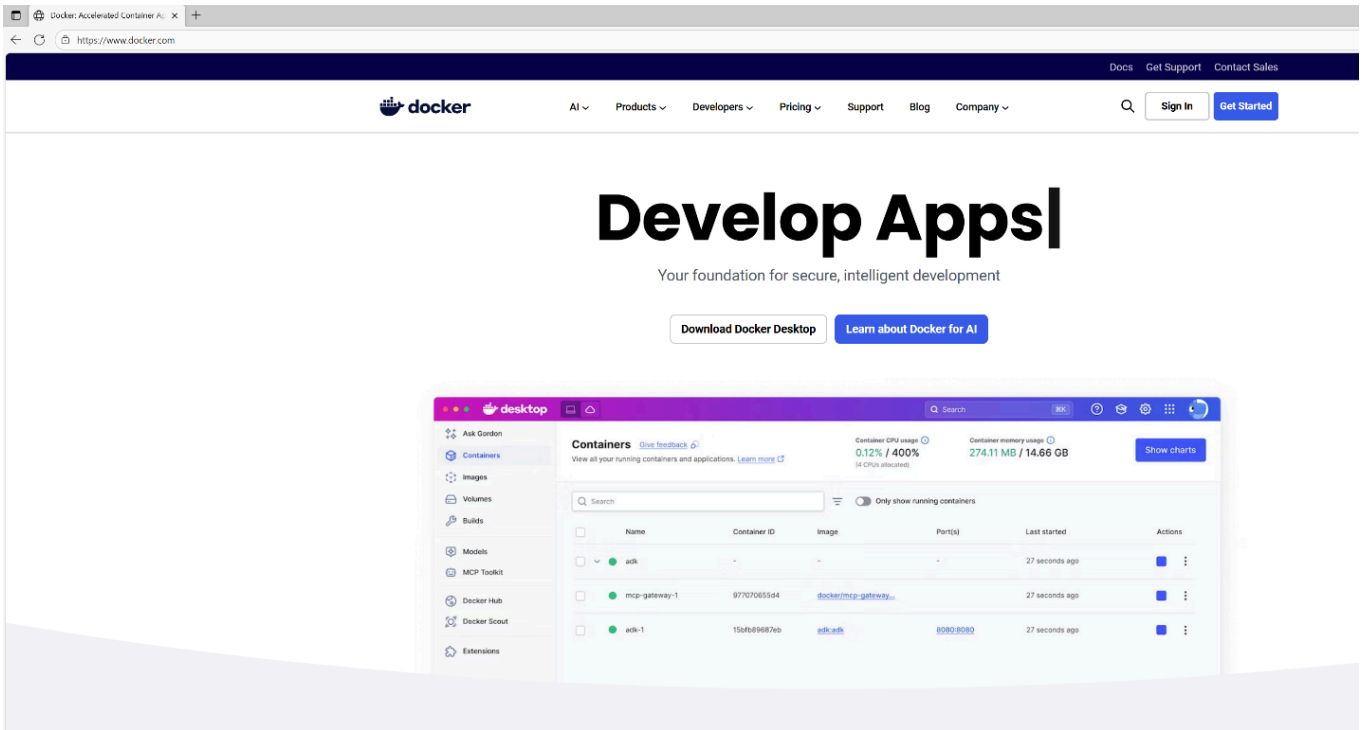


Figure 07. Download Docker Desktop from https://www.docker.com/ while the model setup completes.

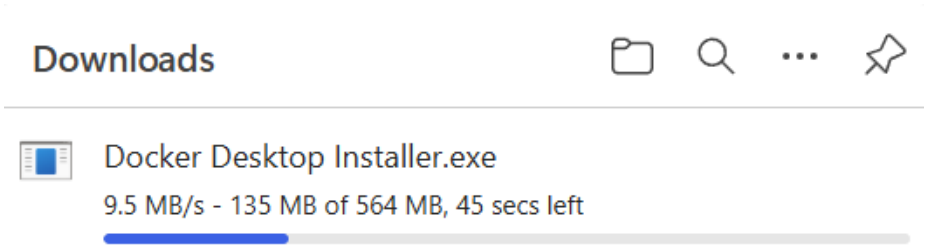


Figure 08. Download Docker Desktop from https://www.docker.com/ while the model setup completes.

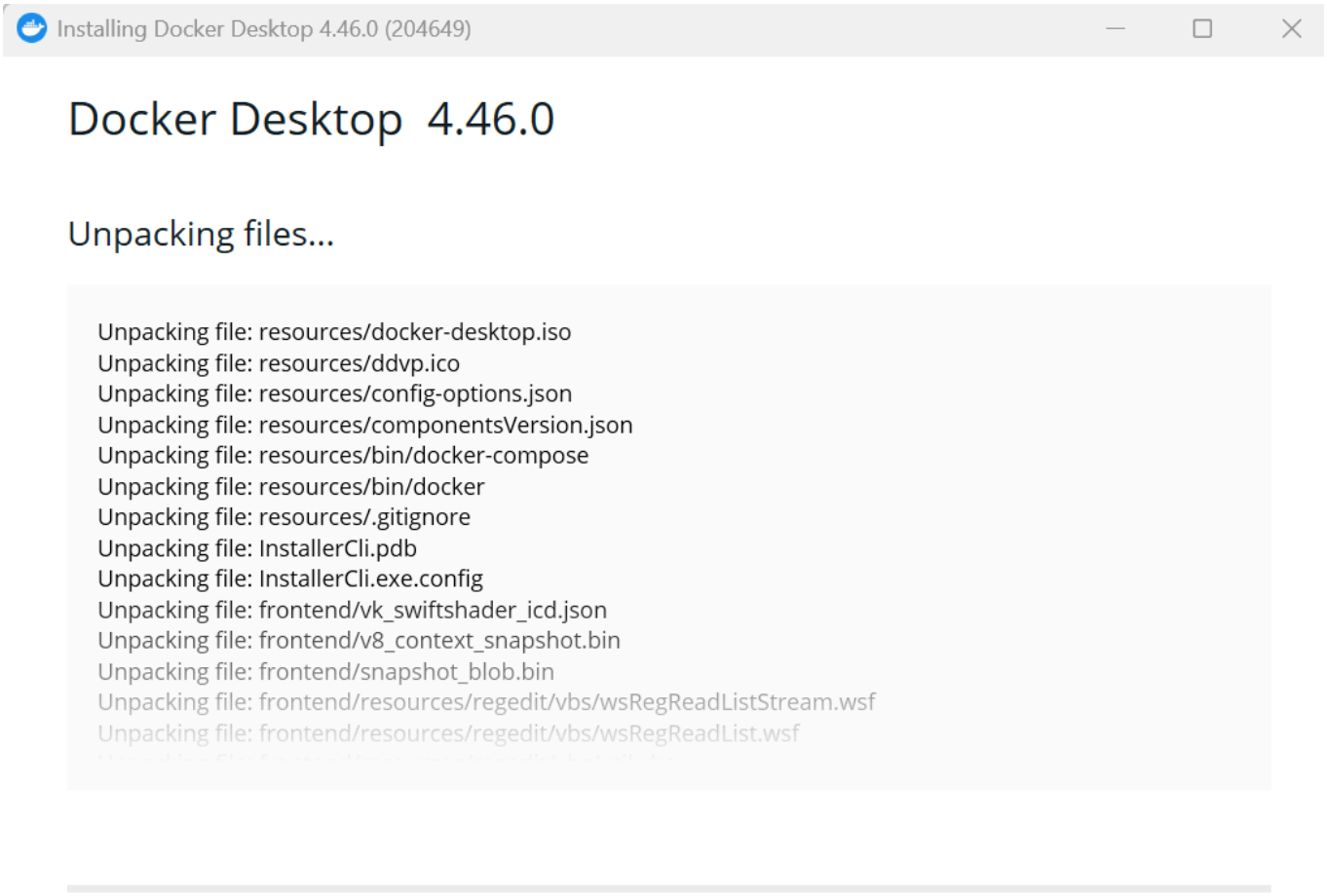


Figure 09. Download Docker Desktop from https://www.docker.com/ while the model setup completes.

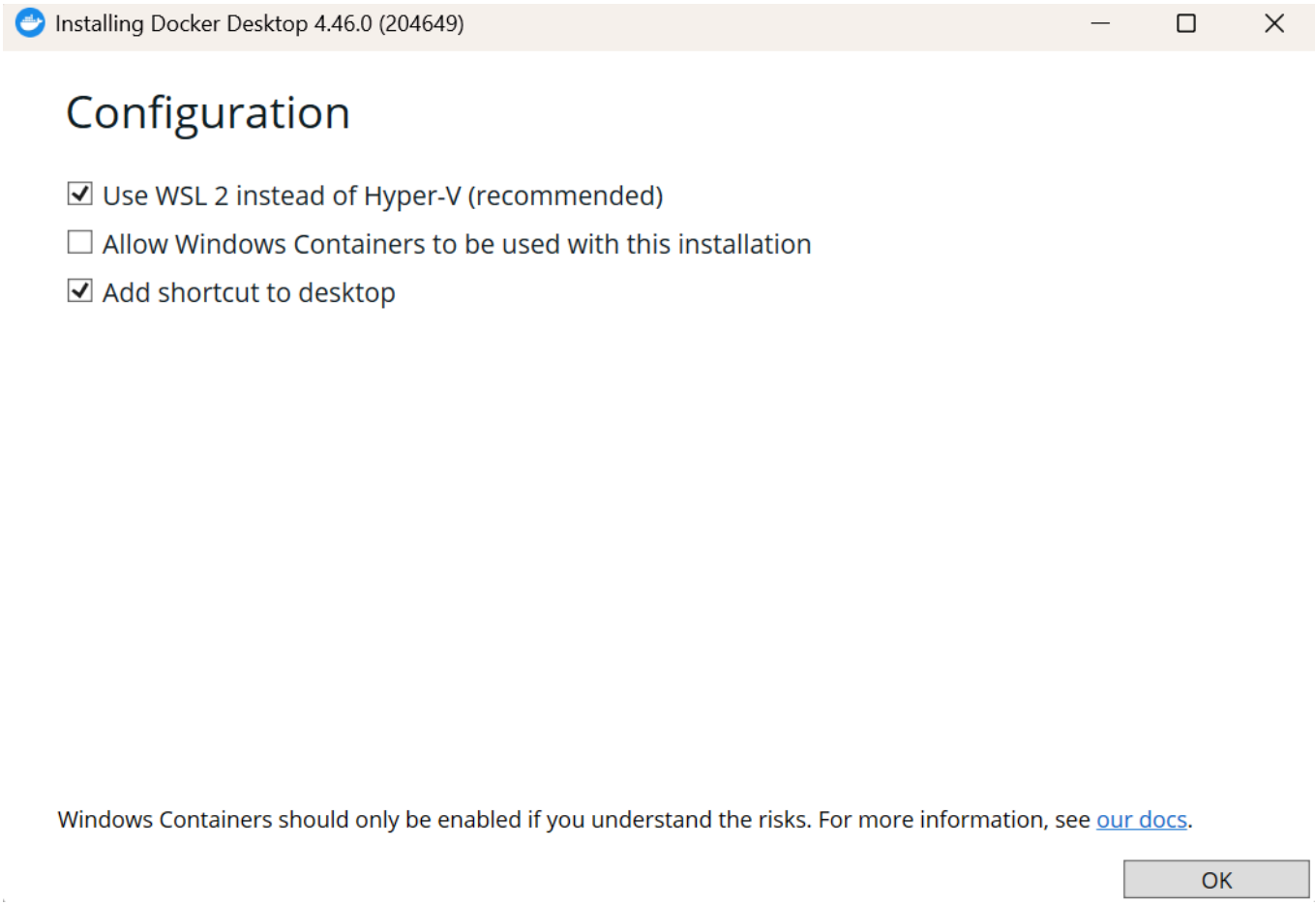


Figure 10. Download Docker Desktop from https://www.docker.com/ while the model setup completes.

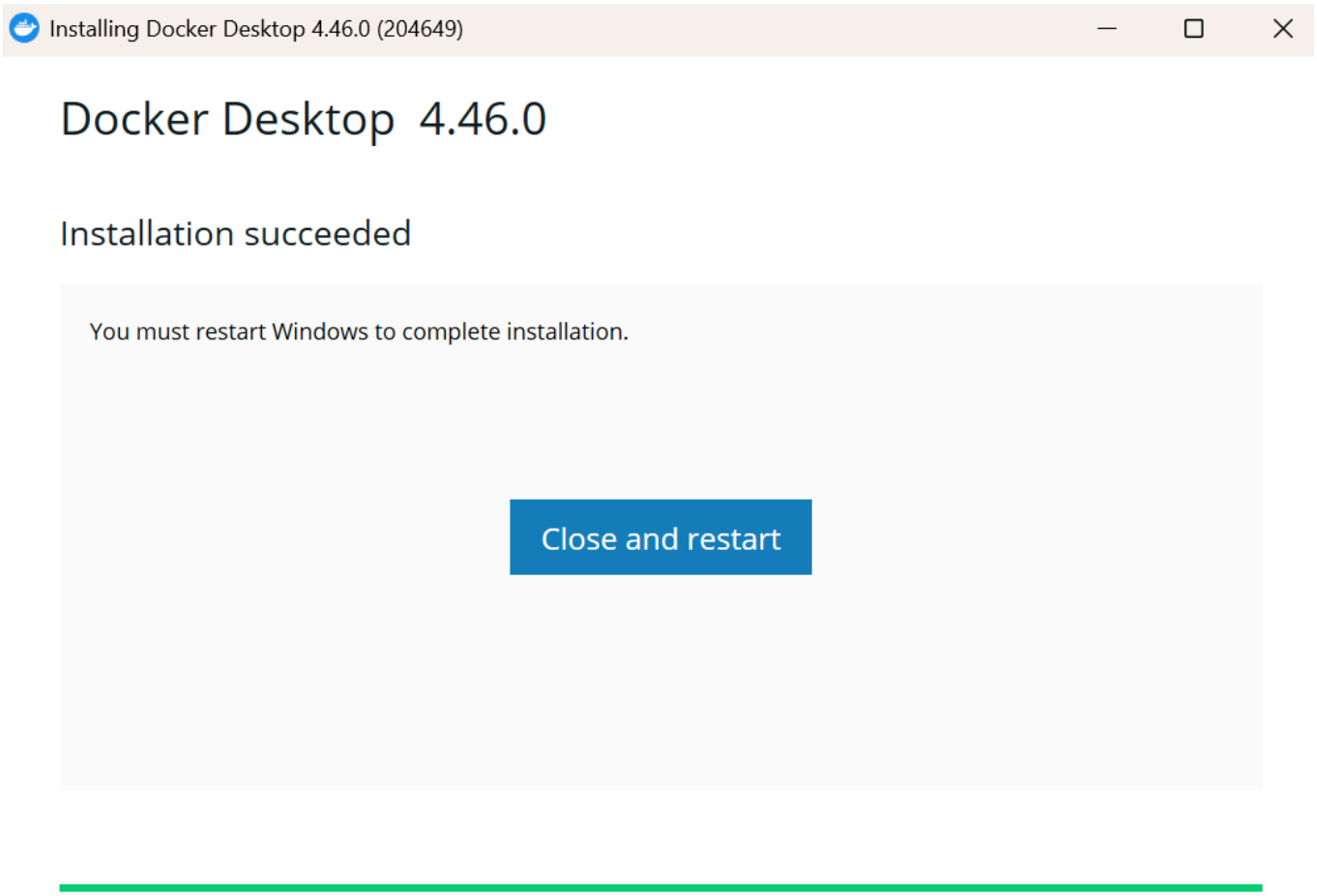


Figure 11. Download Docker Desktop from https://www.docker.com/ while the model setup completes.

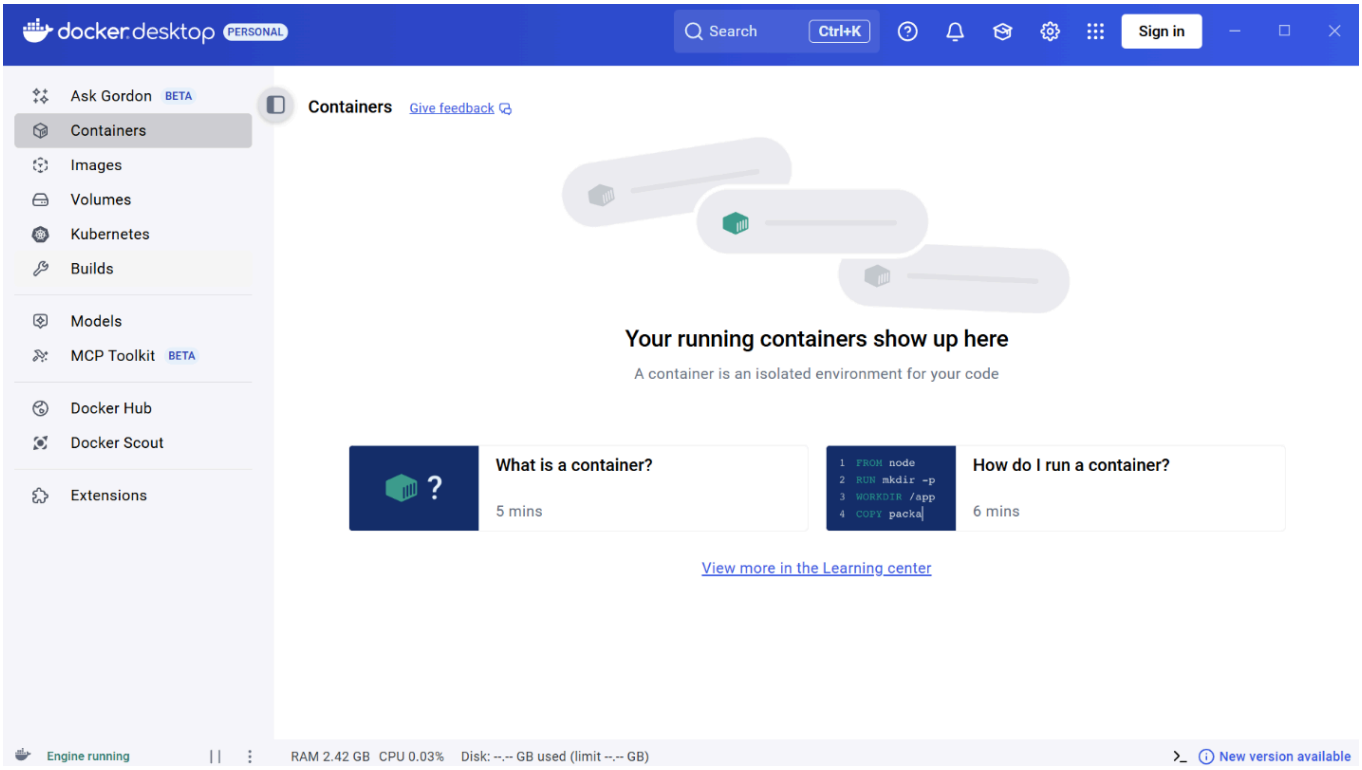


Figure 12. Download Docker Desktop from https://www.docker.com/ while the model setup completes.

Complete Docker Desktop installation and restart the device if prompted. The screenshots record the setup screens shown during the exercise.

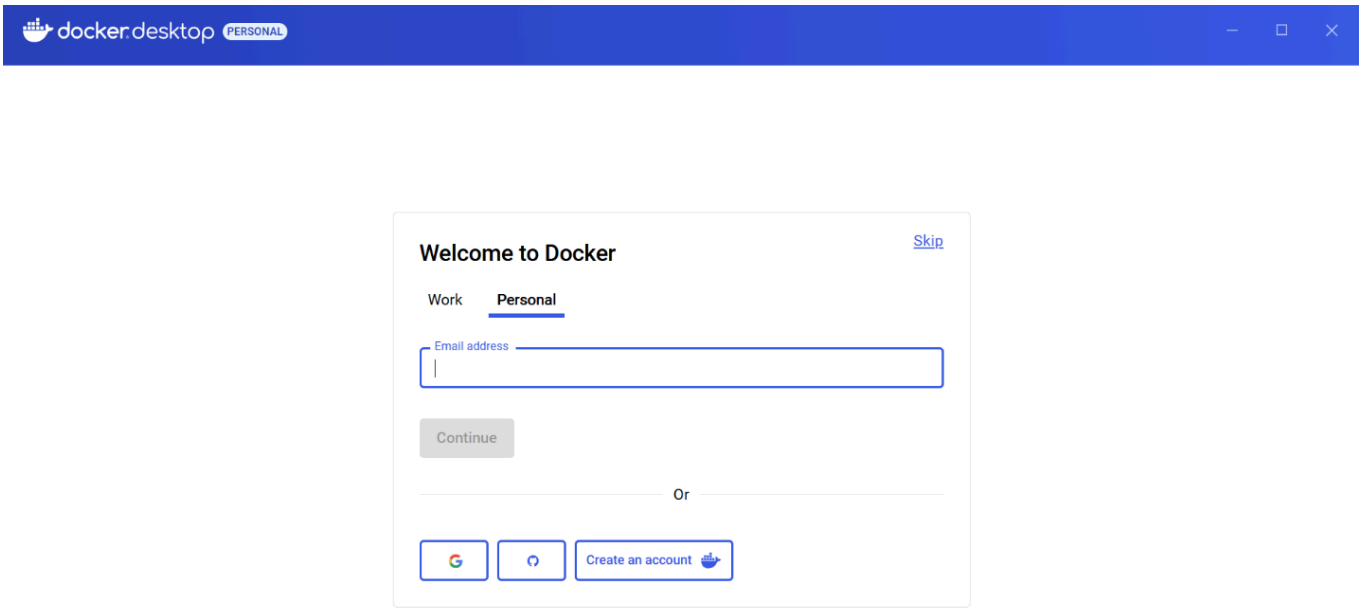


Figure 13. Complete Docker Desktop installation and restart the device if prompted. The screenshots record the setup screens shown during the exercise.

If Docker Desktop reports that WSL needs an update, open PowerShell and update WSL before continuing.

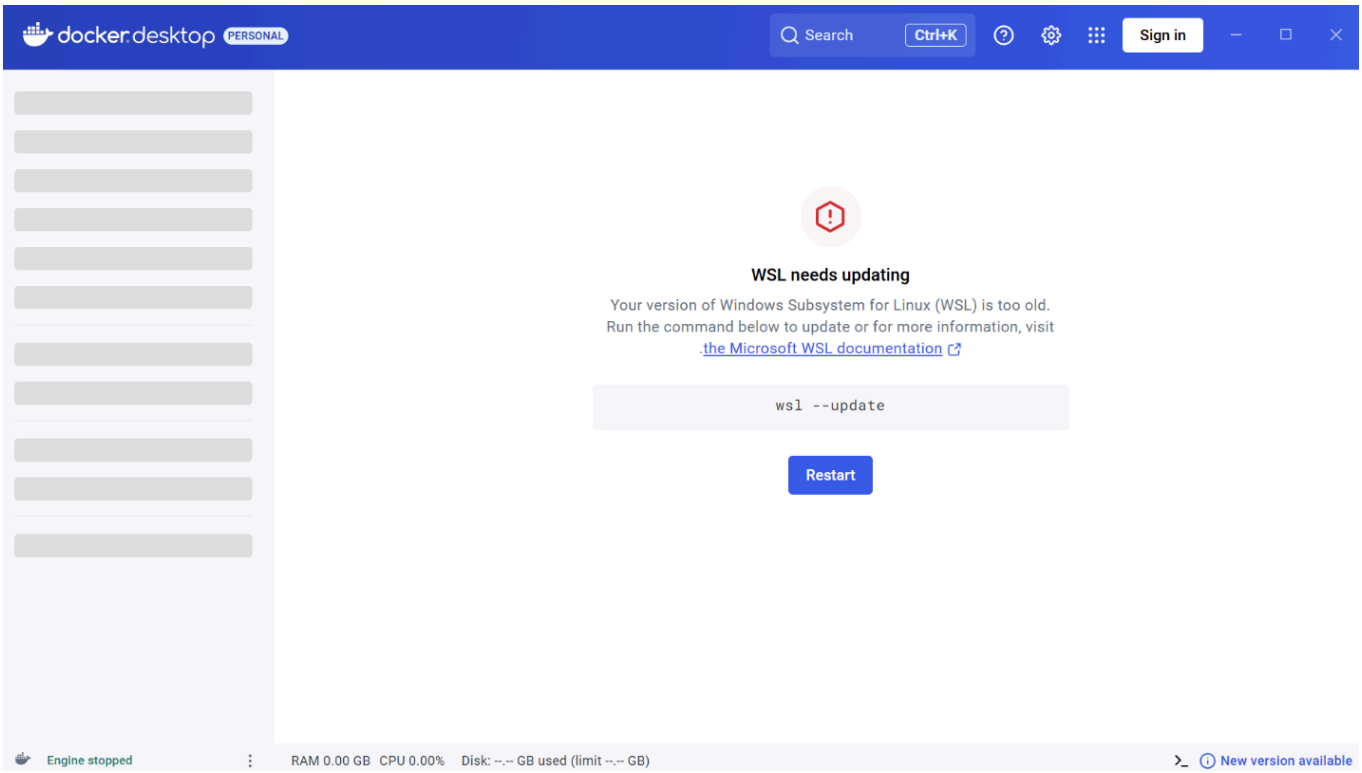


Figure 14. If Docker Desktop reports that WSL needs an update, open PowerShell and update WSL before continuing.

Use the WSL update command: wsl --update. The original report used a single hyphen; the corrected flag is shown here.

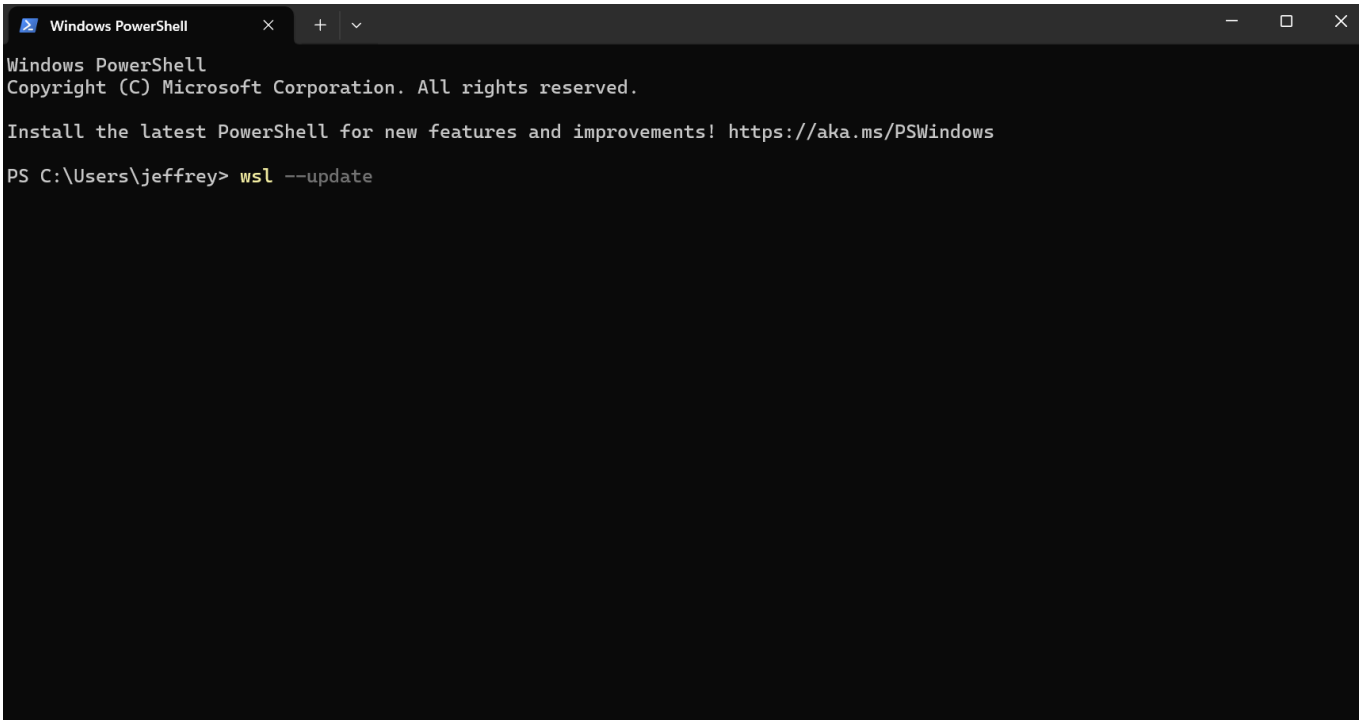


Figure 15. Use the WSL update command: wsl --update. The original report used a single hyphen; the corrected flag is shown here.

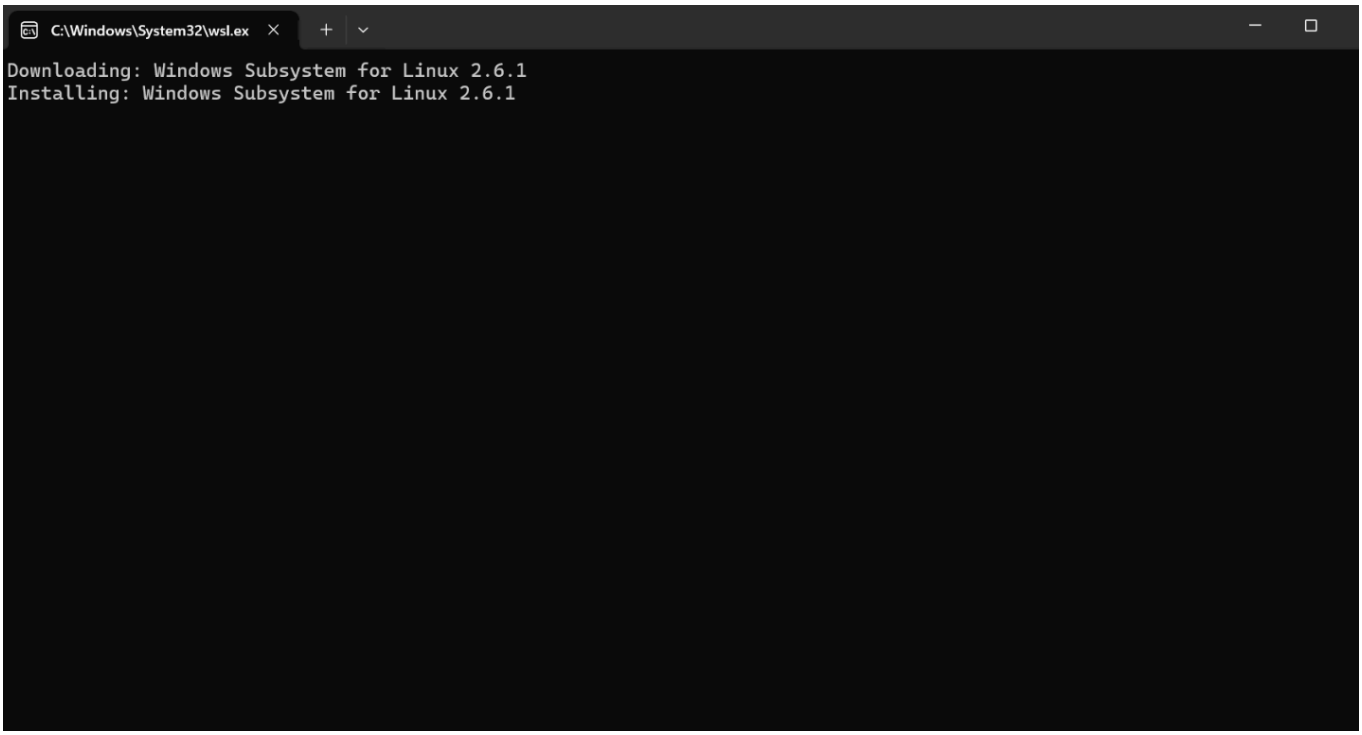


Figure 16. Use the WSL update command: wsl --update. The original report used a single hyphen; the corrected flag is shown here.

The original Windows session below shows PowerShell interaction with Ollama. The corrected reference deployment that follows uses separate CPU-only containers, so it does not depend on the native host service or expose the model API port.

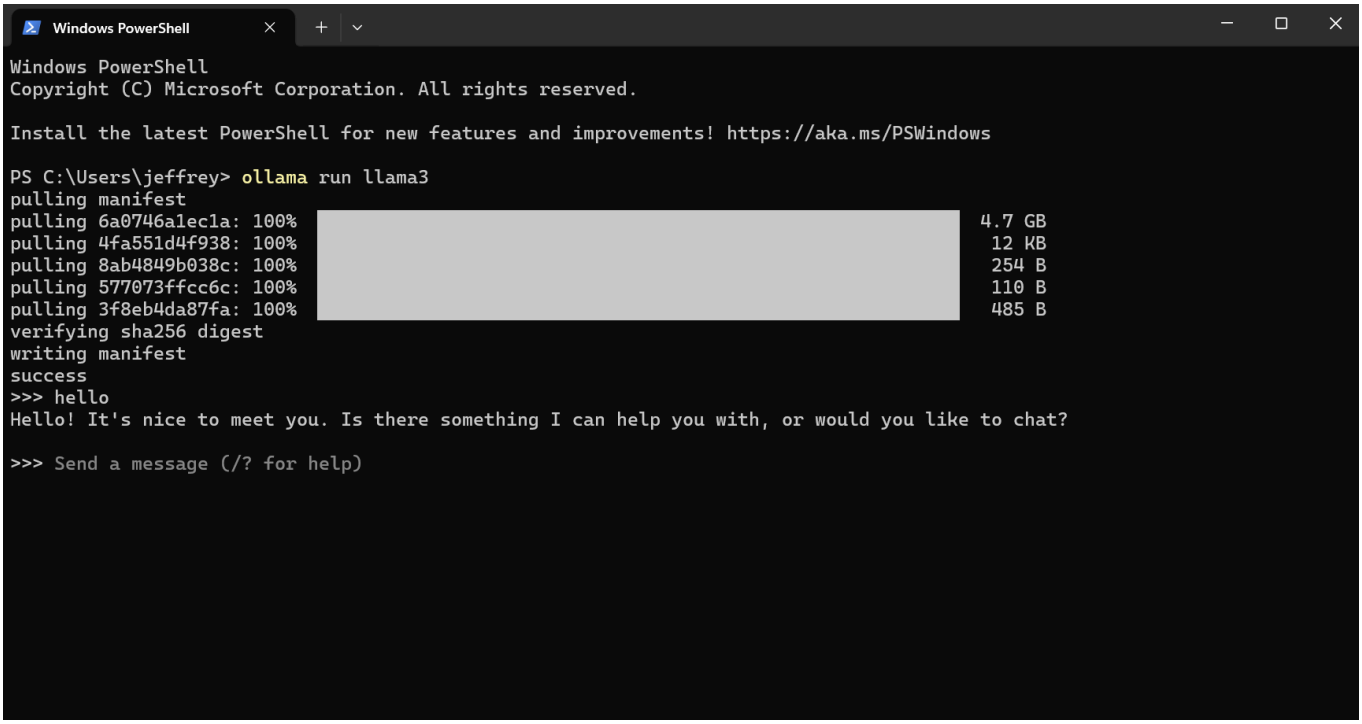


Figure 17. The original Windows session below shows PowerShell interaction with Ollama. The corrected reference deployment that follows uses separate...

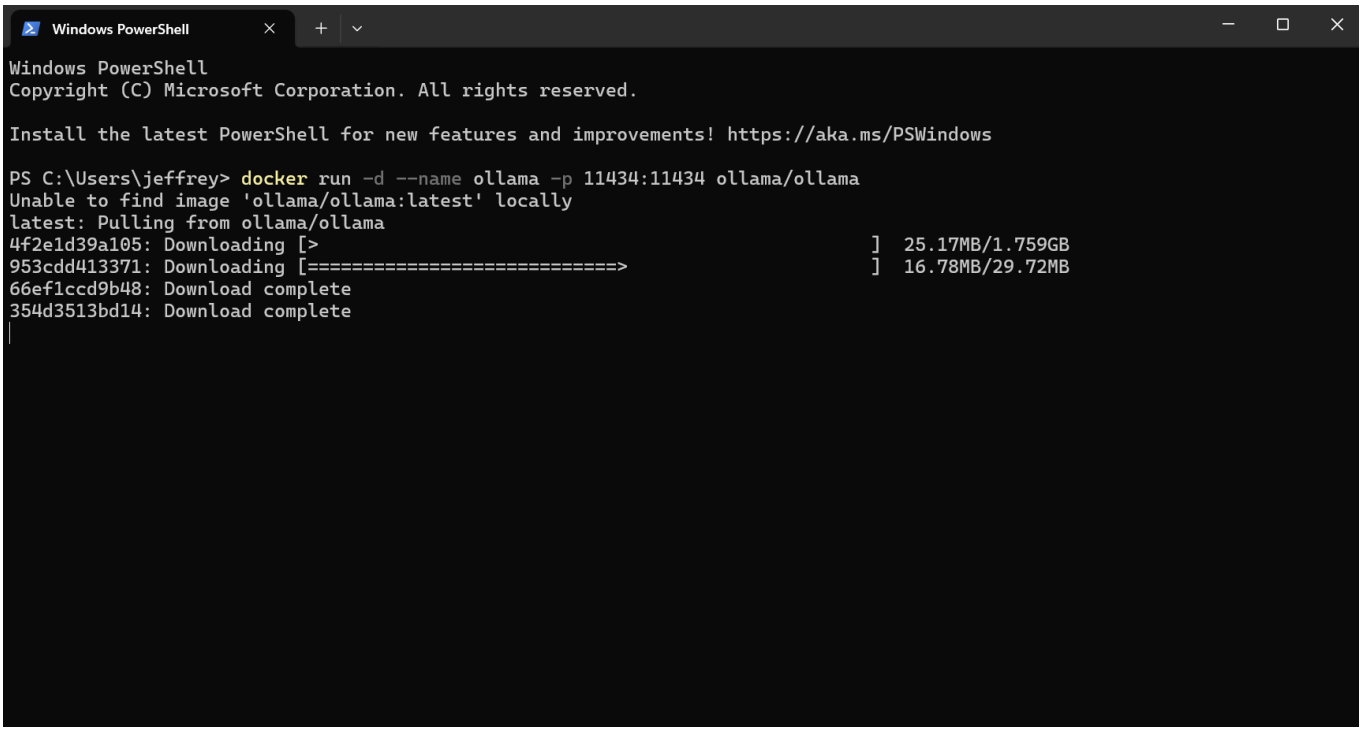


Figure 18. The original Windows session below shows PowerShell interaction with Ollama. The corrected reference deployment that follows uses separate...

Corrected reference deployment: create a Docker network with docker network create local-ai, then start the Ollama container using the complete command below. This is an editorial replacement for the malformed source command; the screenshots still show the original

session.

```
docker run -d --name ollama --network local-ai --restart unless-stopped -v ollama-
data:/root/.ollama ollama/ollama:latest
```

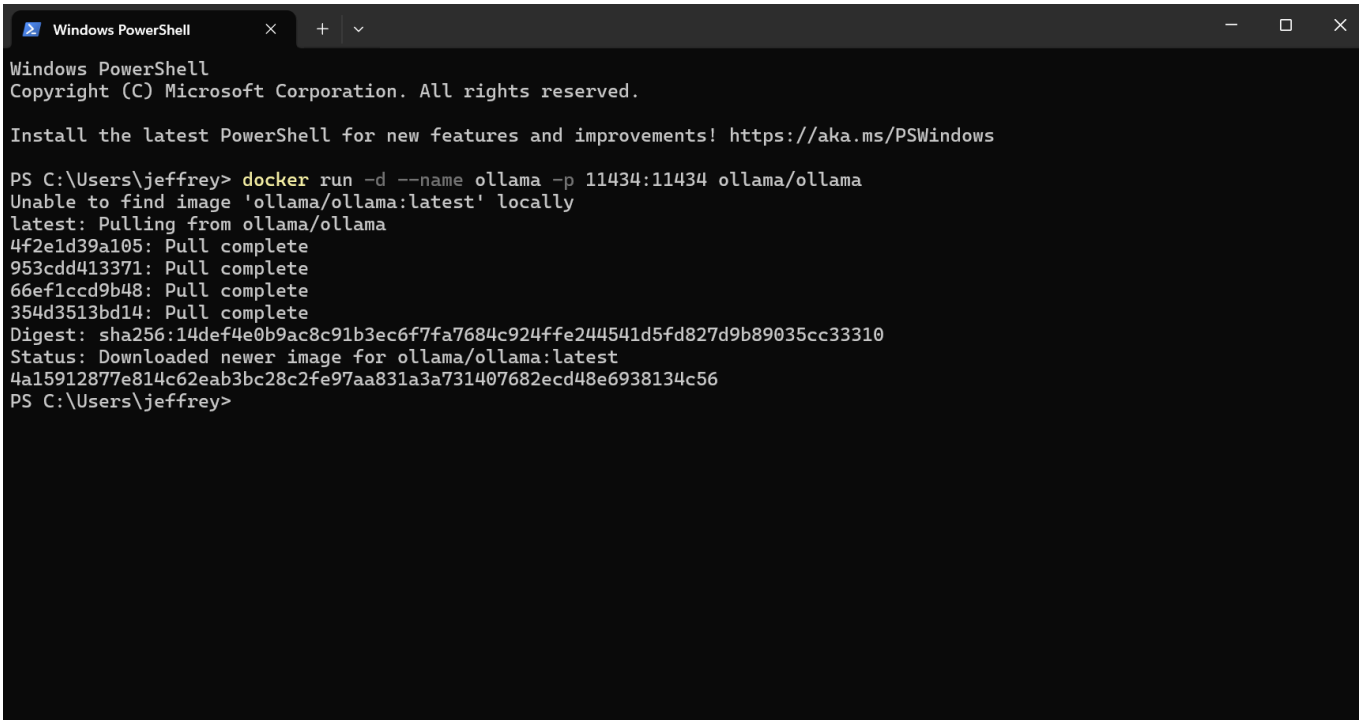


Figure 19. Corrected reference deployment: create a Docker network with docker network create local-ai, then start the Ollama container using the...

After the container is running, download a small example model with docker exec ollama ollama pull llama3.2:1b. The model name is a reference choice rather than a claim about the model used in the original lab. Confirm the installed model with docker exec ollama ollama list.

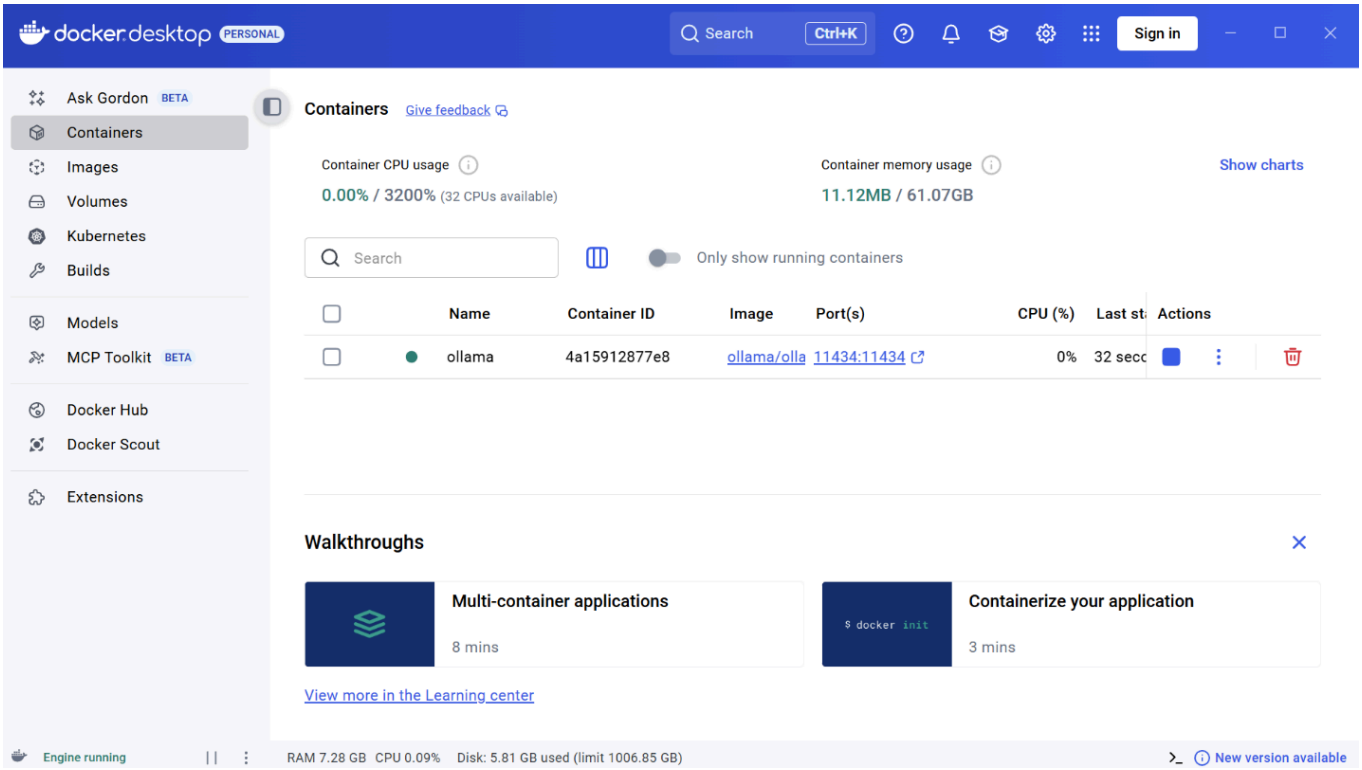


Figure 20. After the container is running, download a small example model with docker exec ollama ollama pull llama3.2:1b. The model name is a...

Start Open WebUI on the same Docker network with an explicit Ollama backend URL, persistent UI storage, and a local HTTP port mapping. The complete command below replaces the source's incomplete example.

```
docker run -d --name open-webui --network local-ai --restart unless-stopped -p 127.0.0.1:3000:8080
-e OLLAMA_BASE_URL=http://ollama:11434 -v open-webui-data:/app/backend/data ghcr.io/open-
webui/open-webui:main
```

```
PS C:\Users\jeffrey> docker run -d -p 3000:8080 --name open-webui --restart=always
```

Figure 21. Start Open WebUI on the same Docker network with an explicit Ollama backend URL, persistent UI storage, and a local HTTP port mapping. The...

Open WebUI connects to http://ollama:11434 through Docker's service-name resolution. The Ollama API is not published on a host port. Allow the services time to start, then verify backend access and model availability using the reference checks near the end of this report.

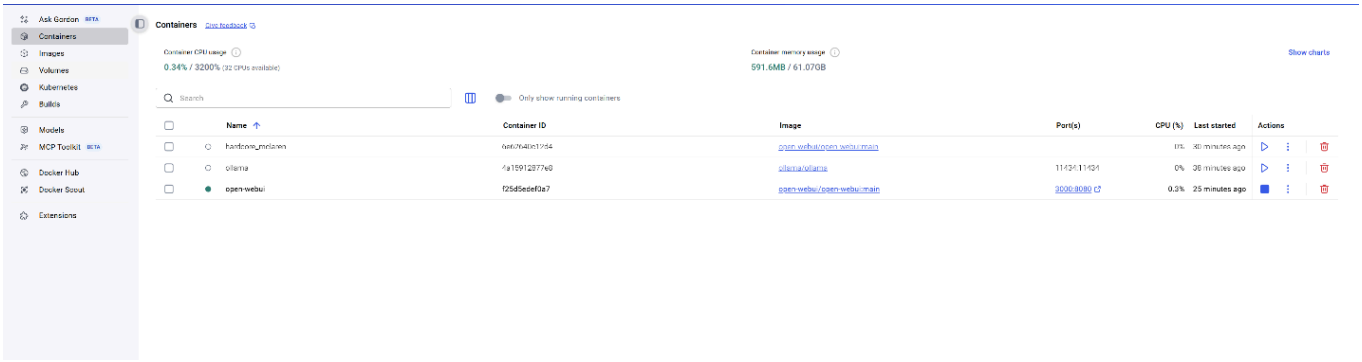


Figure 22. Open WebUI connects to http://ollama:11434 through Docker’s service-name resolution. The Ollama API is not published on a host port. Allow...

Open http://localhost:3000 after Open WebUI finishes starting. Create the initial account if prompted and select the downloaded model. A container port mapping does not configure TLS.

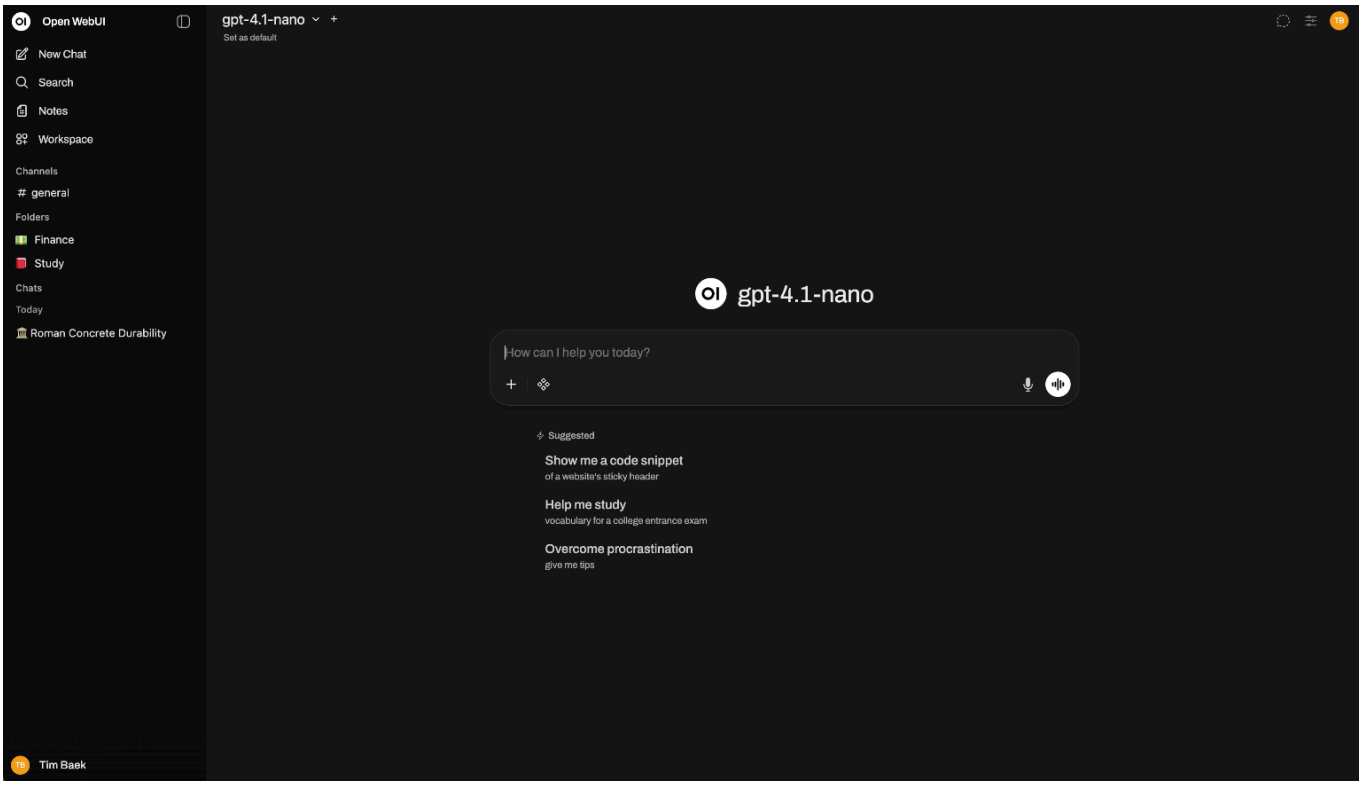


Figure 23. Open http://localhost:3000 after Open WebUI finishes starting. Create the initial account if prompted and select the downloaded model. A...

The final screenshots show optional custom-interface files and development examples. A custom interface requires separate implementation and testing; these images document exploration rather than a production application.

Compose Alternative and Readiness Checks

From the repository root, use examples/local-ai/compose.yaml instead of the separate docker run commands. Choose one deployment method; do not start both on the same UI port. Docker Desktop must be running in Linux-container mode.

```
docker compose -f examples/local-ai/compose.yaml config --quiet
docker compose -f examples/local-ai/compose.yaml up -d
docker compose -f examples/local-ai/compose.yaml exec ollama ollama pull llama3.2:1b
docker compose -f examples/local-ai/compose.yaml exec ollama ollama list
docker compose -f examples/local-ai/compose.yaml ps
```

A Compose dependency controls startup order, not readiness. Retry model download only after Ollama is accepting requests. Check WebUI-to-Ollama connectivity from the UI container using the command below; a nonzero exit status indicates the backend check failed.

```
docker compose -f examples/local-ai/compose.yaml exec open-webui python -c "import urllib.request;
r=urllib.request.urlopen('http://ollama:11434/api/tags', timeout=10); print(r.status);
print(r.read().decode())"
```

Open http://localhost:3000 and submit a short prompt to the selected model. A visible model list and a generated response demonstrate the end-to-end workflow. Inspect docker compose logs if either check fails. Stop this reference deployment with docker compose -f examples/local-ai/compose.yaml down; named volumes remain unless explicitly removed.

The example uses the upstream quick-start image tags (latest and main), which can change. Record image digests when reproducing a particular run. Sources: https://github.com/ollama/ollama and https://github.com/open-webui/open-webui. This example was checked against the upstream documentation and Docker Compose validation, without downloading or running a model.

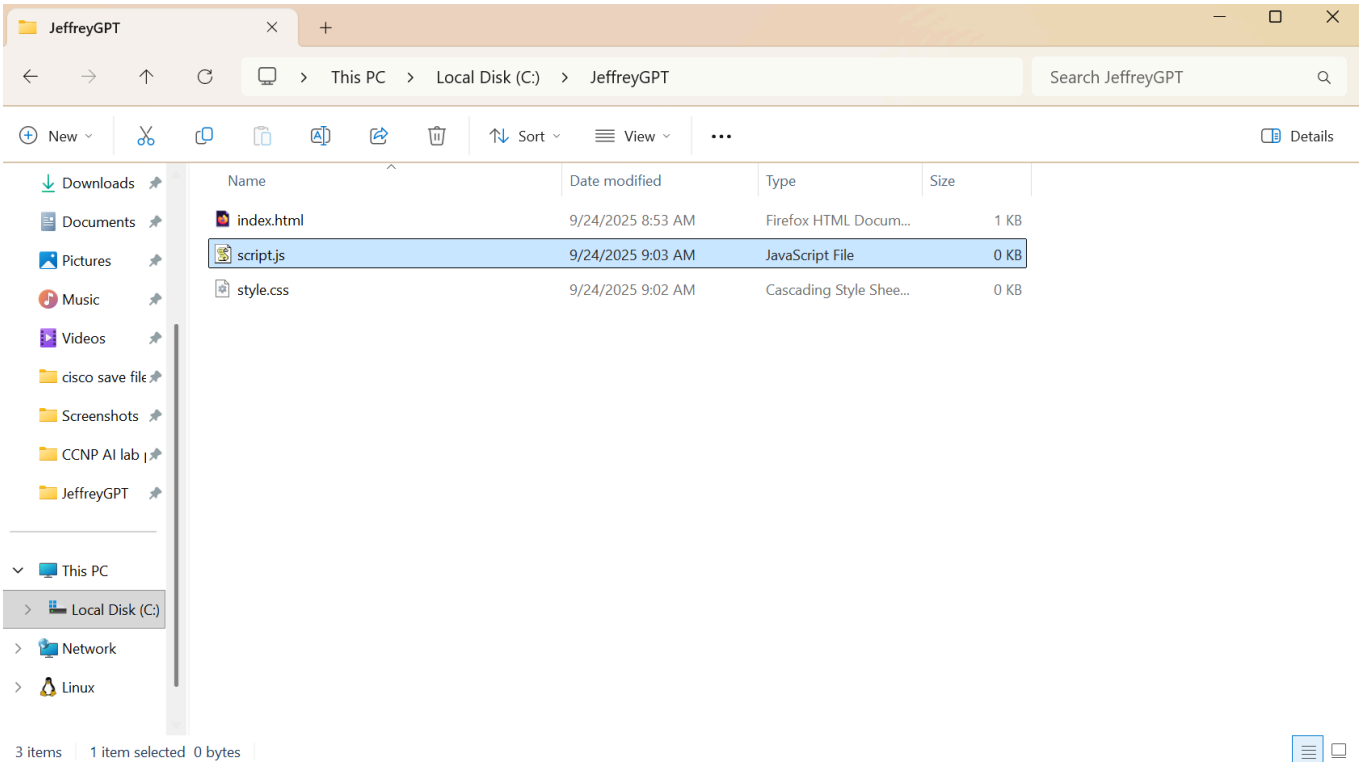


Figure 24. Original custom-interface exploration from the Windows lab session.

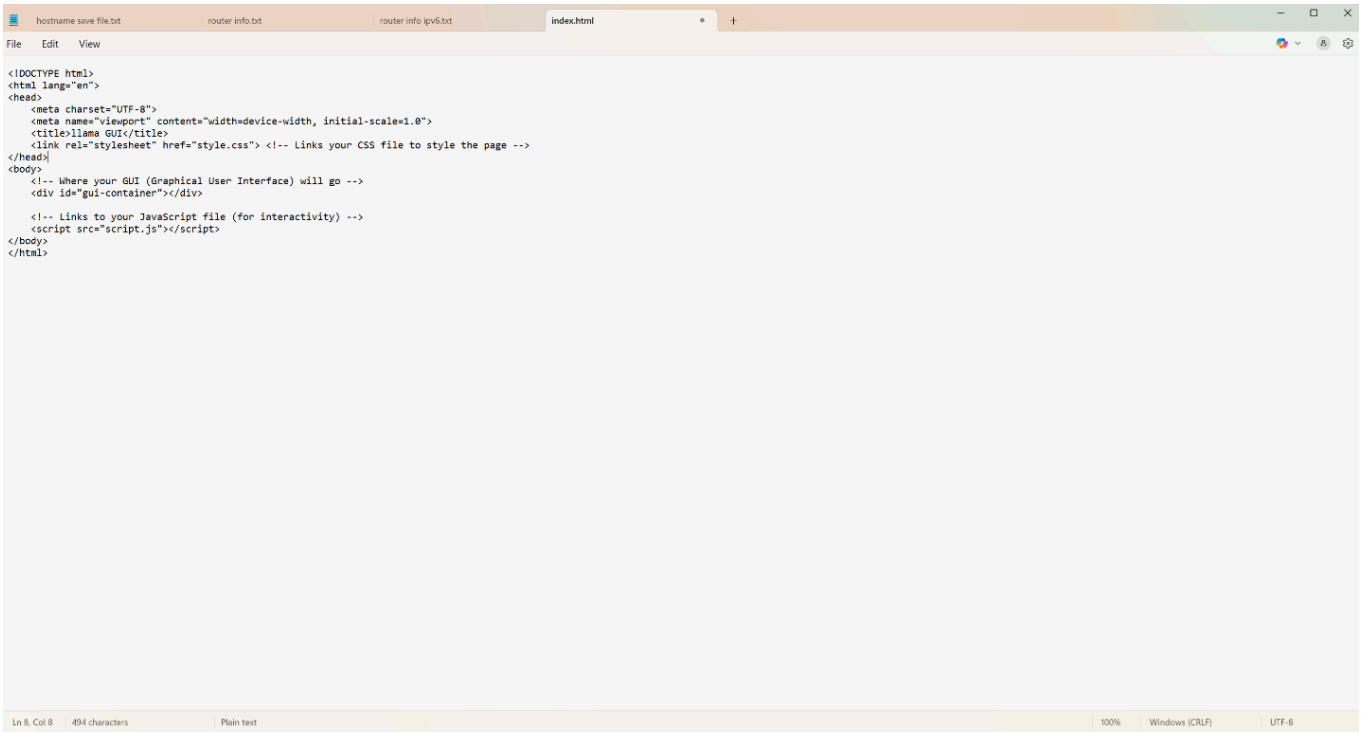


Figure 25. Original custom-interface exploration from the Windows lab session.

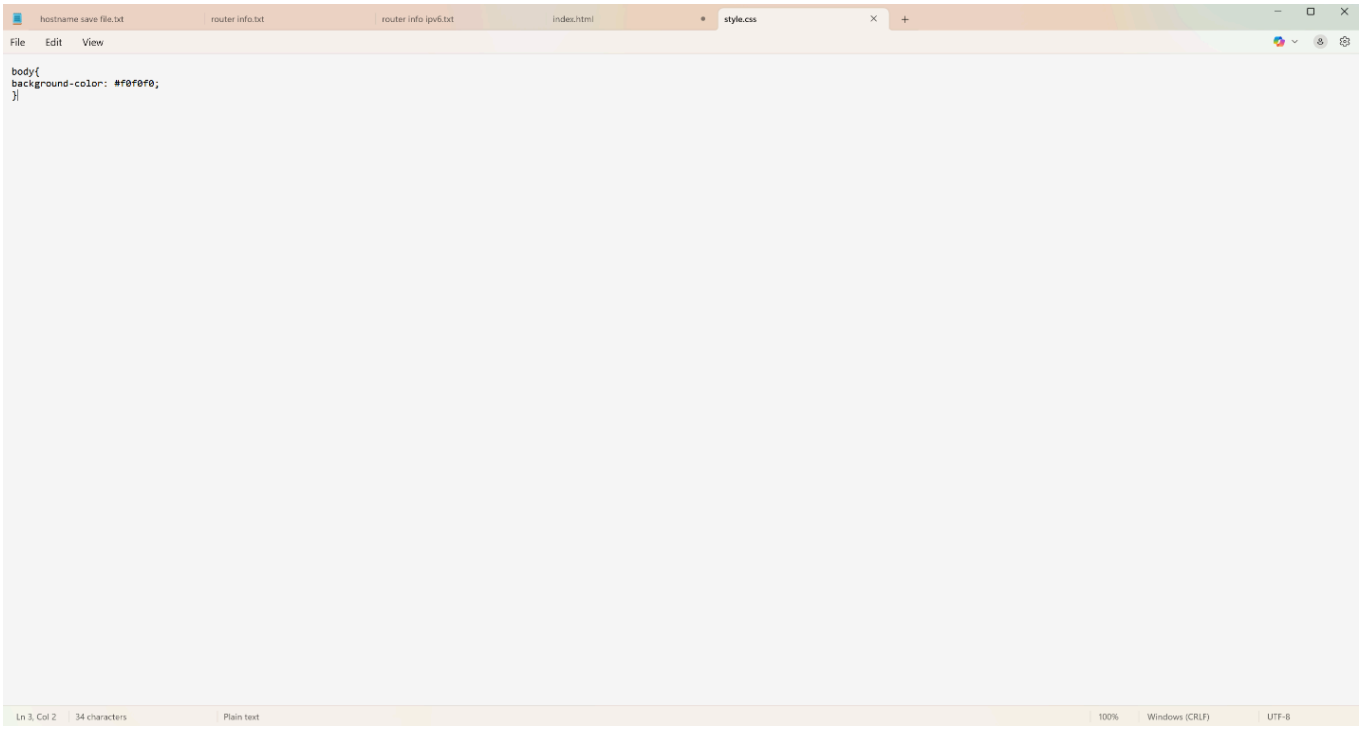


Figure 26. Original custom-interface exploration from the Windows lab session.



Figure 27. Original custom-interface exploration from the Windows lab session.

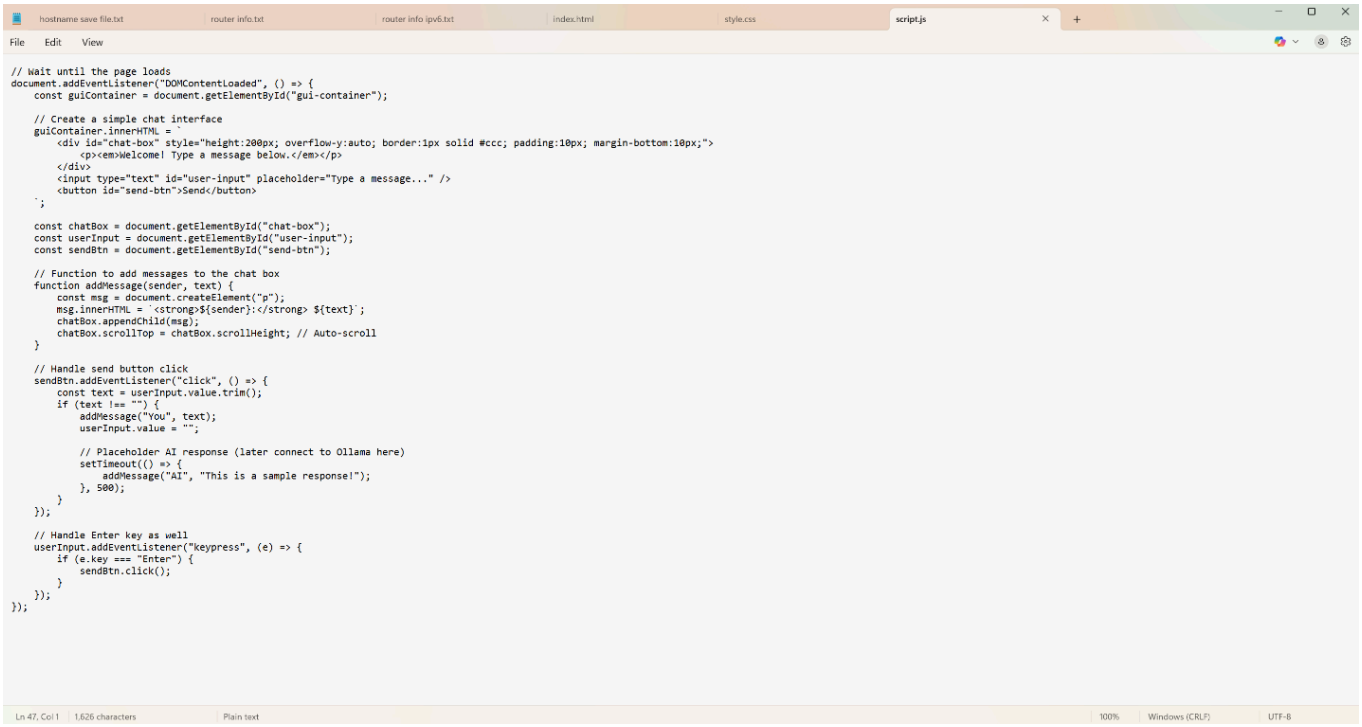


Figure 28. Original custom-interface exploration from the Windows lab session.

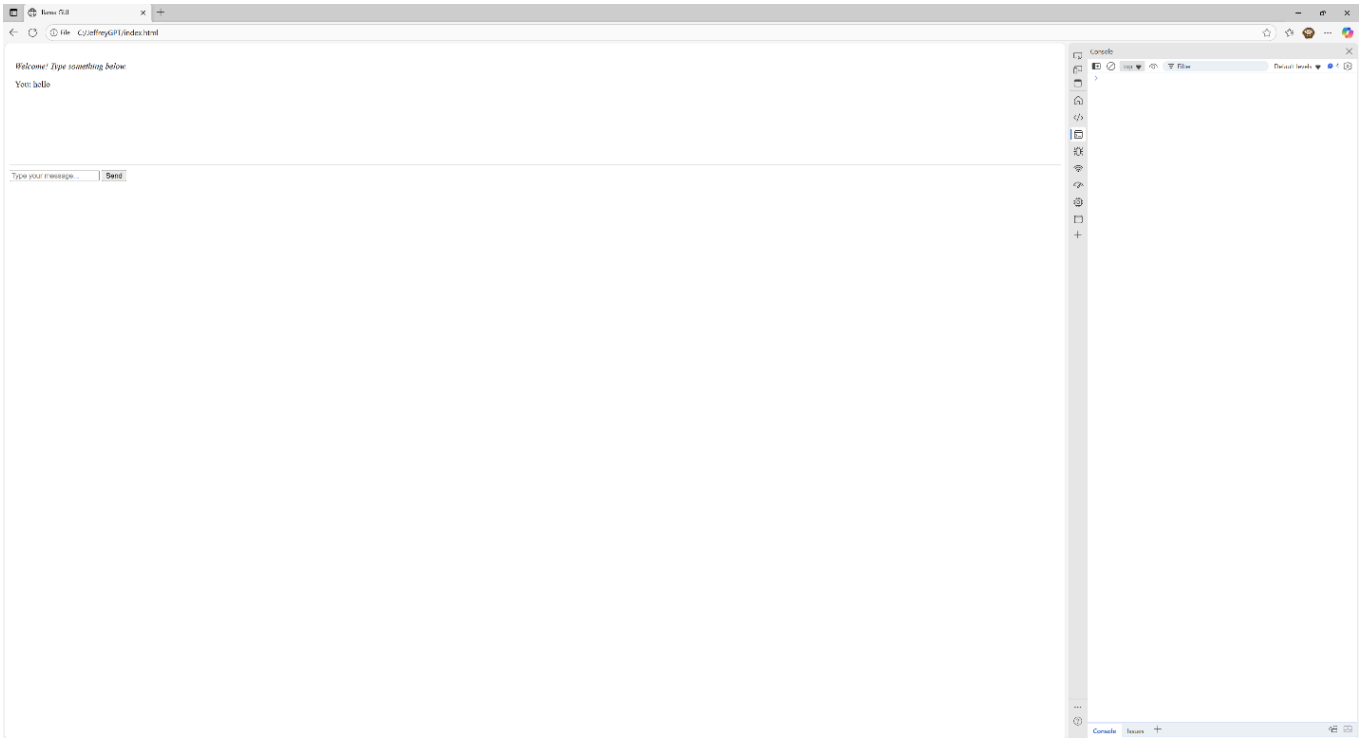


Figure 29. Original custom-interface exploration from the Windows lab session.

Troubleshooting and Lessons Learned

After the initial installations, the team consulted a peer for guidance on the remaining setup. PowerShell and Docker command syntax required additional troubleshooting. Enabling file-name extensions in Windows also helped distinguish HTML files from files saved with a .txt extension.

Conclusion

The original lab introduced local model serving, container management, and a browser interface. The updated reference example provides complete container commands and a Compose alternative. It has been checked for configuration consistency; the model and services were not started during this editing task.